

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

***CO2:** Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15

Total Marks:25

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I Deepan Kumar P certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Deepan Kumar P

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence. (5 Marks)

A Perceptron is the simplest neural network used for binary classification. If a perceptron fails to converge on a linearly separable dataset, it usually indicates problems in learning parameters or implementation.

Possible Causes

- Improper learning rate.
- Incorrect weight initialization.
- Bias not updated correctly.
- Errors in the weight update rule.
- Incorrect data preprocessing.

Debugging Strategies

- Verify the weight update equation.
- Check whether the bias term is updated.
- Monitor training error after each iteration.
- Validate learning rate selection.
- Normalize input features.

Optimization Techniques

- Use a suitable learning rate (0.01–0.1).
- Initialize weights randomly.
- Normalize training data.
- Shuffle training samples before each epoch.
- Stop training when convergence is achieved.

Conclusion

Proper debugging and optimization help the perceptron converge quickly and accurately on linearly separable datasets.

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance. (5 Marks)

Backpropagation is used to train a **Multilayer Perceptron (MLP)** by updating weights to minimize prediction error. Slow convergence usually results from poor parameter settings.

Possible Causes

- Vanishing gradients.
- Poor weight initialization.
- Inappropriate activation functions.
- Incorrect learning rate.
- Unnormalized input data.

Debugging Steps

- ✓ **Monitor loss after every epoch.**
- ✓ **Verify gradient calculations.**
- ✓ **Check activation function outputs.**
- ✓ **Compare training and validation errors.**
- ✓ **Ensure data normalization.**

Optimization Techniques

- ❖ Use ReLU activation.
- ❖ Apply Adam Optimizer.
- ❖ Normalize input features.
- ❖ Use Xavier or He weight initialization.

- ❖ Apply learning rate scheduling.

Conclusion

Proper debugging combined with modern optimization methods improves convergence speed and prediction accuracy.

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed. (5 Marks)

A **Genetic Algorithm (GA)** is an optimization technique inspired by natural evolution. Poor performance usually occurs due to incorrect chromosome representation or loss of diversity.

Possible Causes

- Improper chromosome encoding.
- Incorrect fitness function.
- Premature convergence.
- Low mutation rate.
- Small population size.

Debugging Steps

- Verify chromosome representation.
- Check fitness calculations.
- Monitor diversity in each generation.
- Observe crossover and mutation operations.
- Compare fitness values across generations.

Optimization Techniques

- Increase mutation rate moderately.

- Apply elitism.
- Use tournament selection.
- Increase population size.
- Adjust crossover probability.

Conclusion

Proper parameter tuning improves exploration and produces high-quality optimal solutions.

4. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization. (5 Marks)

Introduction

A **Decision Tree** is a supervised learning algorithm used for classification and regression. High training accuracy but poor testing accuracy indicates overfitting.

Possible Causes

- Deep tree structure.
- Overfitting.
- Noisy data.
- Irrelevant features.
- Small training dataset.

Debugging Steps

- Compare training and testing accuracy.
- Examine tree depth.
- Identify noisy attributes.
- Perform cross-validation.
- Evaluate feature importance.

Optimization Techniques

- Apply pruning.
- Use cross-validation.
- Select important features.
- Limit maximum tree depth.
- Remove noisy data.

Conclusion

Pruning and feature selection improve generalization and reduce overfitting.

5. A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling. (5 Marks)

A **Support Vector Machine (SVM)** is a supervised machine learning algorithm used for classification and regression tasks. It works by finding the optimal hyperplane that separates different classes. However, when an SVM model is applied to a **nonlinear dataset**, its performance may decrease due to an inappropriate kernel, poor parameter tuning, or improper feature scaling.

Issues Affecting SVM Performance

The following factors may cause poor performance in a nonlinear dataset:

- **Incorrect Kernel Selection:** Using a linear kernel for nonlinear data cannot capture complex relationships.
- **Improper Hyperparameter Tuning:** Incorrect values of **C** and **Gamma** may lead to overfitting or underfitting.
- **Feature Scaling Problems:** Features with different ranges affect the distance calculations used by SVM.
- **Presence of Noise and Outliers:** Noisy data can reduce classification accuracy.

Debugging Techniques

To identify and solve the problem, the following debugging steps should be performed:

1. Examine the dataset to determine whether it is linear or nonlinear.
2. Test different kernel functions such as **Linear**, **Polynomial**, and **Radial Basis Function (RBF)**.
3. Check the values of **C** and **Gamma** and observe their effect on accuracy.
4. Normalize or standardize all input features before training.
5. Compare training accuracy and testing accuracy to detect overfitting or underfitting.

Optimization Methods

The following optimization techniques improve the performance of the SVM model:

- **Kernel Selection:** Use the **RBF kernel** for nonlinear datasets because it can model complex decision boundaries.
- **Hyperparameter Tuning:** Apply **Grid Search** or **Random Search** to find the optimal values of **C** and **Gamma**.
- **Feature Scaling:** Standardize the dataset using **StandardScaler** or normalize it using **MinMaxScaler**.
- **Cross-Validation:** Use **K-Fold Cross-Validation** to evaluate the model on different data splits and improve generalization.

Conclusion

The performance of an SVM on a nonlinear dataset depends on selecting the appropriate kernel, tuning hyperparameters, and applying proper feature scaling. By systematically debugging these issues and using optimization techniques such as **RBF kernel selection, Grid Search, feature scaling, and cross-validation**, the SVM model can achieve higher accuracy, better generalization, and improved classification performance.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	30	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	10	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand